AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_C_task_9.pdf Version : V1.0	

LOW LEVEL C-PROGRAMMING AND MICROCOMPUTER ARCHITECTURE – C_task_9

Darwin Rengifo
2018-03-11

Sammanfattning: Det kapitel prata om olika typer CPU arkitektur och på detta CASE ska vi jobba med två typer av arkitektur. Den HW_CASE_4 visa en enklare CPU arkitektur och HW_CASE_4_F visa en komplex arkitektur. Det pratar om även cache minne och hur använder CPU för att reducera tillgång tiden till huvud minne. Det kapitel även visa hur accelerera utförandet av instruktionerna implementerande en form av parallellism inne av en enda CPU igenom av "Pipeline" instruktionen.
Denna länk: https://www.altera.com/products/processors/support.html; det visar dokumentering som ha NIOS II Processor för att utveckla hårdvara, programvara. Det visar "Platfrom Designer" dokumentation för att genererar automatiskt sammankopplingslogisken för att koppla funktioner och delsystem av intellektuell ägande.
Ladda ned av produkter NIOS II och licenser av verktygen som ska använda.

Innehåll

LOW LEVEL C-PROGRAMMING AND MICROCOMPUTER ARCHITECTURE – C_task_9.....1

1. INLEDNING.....3
2. KRAVSPECIFIKATION.....3
3. BESKRIVNIN AV FÖLJANDE BEGREPP OCH HUR DE PÅVERKAR PRESTANDA(övergripande):.....4
4. BESKRIVNING AV TVÅ OLIKA HW-ARKITEKTURERNA.....5
5. ANVÄNDNING TEST PROGRAMMET I BILAGA A.....5
6. TESTFALLEN ATT SKA ANVÄNDAMED MED DE TVÅ OLIKA PROCESSORERNA(economy och fast).....6
7. TABELL FÖR TESTFALL.....6
8. KORFATTAD ANALYS AV SIFFRORNA.....7
9. BILAGA.....8

1. INLEDNING

Denna rapport beskriver stegen för att mät exekveringstid av ett program på två olika HW-arkitekturerna. För att göra detta, i rapporten det finns testfallen(se tabell 1) som använder till testa prestanda av programmet när utför med de två olika processorerna.

2. KRAVSPECIFIKATION

Krav	Beskrivning	Utfört Ja/nej
	Förstudie	
Krav_001	Gå igenom teorilektionen (Teori_9 och länk*) och skriv en kort sammanfattning vilken ska ingå i rapporten enligt krav_007 (eget kapitel). Beskriv följande begrepp och hur de påverkar prestanda (övergripande): - Data and instruction cash - Pipeline - Beskriv gärna andra CPU-mekanismer - Svara på frågan; Vad är det för skillnad när det gäller prestanda (accesstid) på internal RAM och external RAM (utanför chippet)? *https://www.altera.com/products/processors/support.html	JA
Krav_002	Beskriv de olika HW-arkitekturerna (eget kapitel) - 50MHz economy (HW_CASE_4) - 50 MHz fast (HW_CASE_4_F)	JA
	Funktionskrav	
Krav_003	Använd testprogrammet i bilaga A.	JA
Krav_004	Följande testfall ska gås igenom med de två olika processorerna (economy och fast): ***Testfall 1*** **Mjukvara (all programvara)**: Interna FPGA RAM, enable_reduced_device_drivers, enable_small_c_library. ***Testfall 2*** **Mjukvara (all programvara)**: SRAM minnet/Ingen C-kods optimering ***Testfall 3***	JA

	Mjukvara (all programvara): SRAM minnet/ enable_reduced_device_drivers, enable_small_c_library.	
Krav_005	Tabell 2 enligt bilaga B ska fyllas i med antal klockcykler och tid. Lägg gärna till % av förändring.	JA
Krav_006	Gör kortfattad analys av siffrorna. (eget kapitel)	JA
Dokumentationskrav		
Krav_007	Sammanfoga dokumentationen från krav_001 till krav_006 till en läsbar rapport med separata kapitel. Framsida med titel, en kort sammanfattning, innehålls-förteckning och kapitel. Lägg även till eventuella slutsatser och referenser.	JA
Leveranskrav		
Krav_008	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en rapport. Namnet på filen ska vara ”fornamn_efternamn_C_task_9”. Sista leveransdag se kursschema (för VG).	JA

3. BESKRIVNIN AV FÖLJANDE BEGREPP OCH HUR DE PÅVERKAR PRESTANDA(övergripande):

DATA AND INSTRUCTION CASH: Data cache har som uppgift att sparar data som använder ofta i CPU. Instruktion cache har som uppgift att sparar instruktionerna som använder ofta i CPU. Den påverka normalt i prestanda eftersom CPU använder cache minne för att reducera tillgång tiden genomsnitt till huvudminnet.

PIPELINE: Den är en serie av elementen av ”data processing” som är ordnade på ett sådant sätt att utgången av varje element är ingången av nästa element. Den orienterat till ”data processing” och instruktioner. Den påverka normalt i prestanda eftersom det möjliggör en snabbare CPU-prestanda vid en given klockhastighet.

BESKRIVNING AV ANDRA CPU-MEKANISMER:
ALU(Arithmetic Logic Unit): Den här mekanismen har som syften få fram binära data sparade på minnet och utföra matematiska/logiska operationer med detta data enlighet med instruktioner som kommer av CU(Control Unit).

CU(Control Unit): Den drar instruktioner av minnet sen avkodas och utföra dom, det kommer att kallar till ALU när som behöver. Producerar och använder alla kontrollsignaler som behövs för att synkronisera operationerna och dataöverföringar innanför och utanför ALU. Den påverka normalt i prestanda eftersom målet av CU är att sekvensera funktionen av hela systemet.

Databussen: Den används för att transporterar data mellan mikroprocessorn och perifert, både på etta som på andra ledden(<---->) och deras storlek(linjer antalen),

är det värdet som definierar antalet bits av mikroprocessor och motsvarar antalet bits att kan hanterar på samma gång ALU. Den påverka normalt i prestanda eftersom databussen definierar sin kapacitet enligt maximal frekvens av sändnings och data bredden.

Adressbuss: Den kan användas för att kommunicerar mikroprocessor med cellerna av RAM minne. När mikroprocessor vill läsa cellen innehåll, den skickar via adressbussen cellen som vill läsa (varje cell på minne har sin egen adress) och få cellen innehållet genom databussen. Den påverka normalt i prestanda eftersom addressbussen storlek definierar kvantitet av RAM minne att kan hanterar mikroprocessor.

Kontrollbuss: Det används för att kontrollerar databussen och adressbussen. Organiserar och skickar informationen mot lämplig buss enligt informationen som vill överför. En kontrollbuss måste ha som minst 10 eller mer linjer för att var nyttig eftersom sådär kan det skickar mer information bits samtidigt. Bussarna beter sig som autentiska kommunikationslinjer. Den påverka normalt i prestanda eftersom en effektiv design av buss är fundamental till att systemhastighet.

VAD ÄR DET FÖR SKILLNAD NÄR DET GÄLLER PRESTANDA(accesstid) PÅ INTERNAL RAM OCH EXTERNAL RAM(utanför chippet)?
Skillnaden av använder interna RAM, är att utföring tiden är mer snabbare som externa RAM, detta är möjlig med en bra programvara optimering och design en bra hårdvara arkitektur.

4. BESKRIVNING AV TVÅ OLIKA HW-ARKITEKTURERNA

50MHz economy(HW_CASE_4): Denna arkitektur är utforma för att uppnår minsta möjliga kärnan(core, kernel) storlek. Detta är en IP-komponent som innehålla en HAL mapp till programvaran(det kan se på system.h arkiv som innehålla hela hårdvaran och som vardera av hårdvaran komponenter kopplade till Avalon buss, det kommer att ha en enda address.), en HDL mappen för att hårdvaran. Den har inte en (pipeline), den som betyder att alla instruktionen kommer att ha runt av 6 klockcykler för att slutför.

50MHz fast (HW_CASE_4_F): Denna arkitektur är utforma för att uppnår en hög utförande prestanda. Prestanda är erhåller på bekostnad av kärnan(core, kernel) storlek på chipet. Det har 6 pipeline etapper, som betyder att ha samma "latency", men prestanda kommer att bli 1 klockcykel. De 6 etapper är implementerar på 6 specialiserade hårdvaran enheter alltså den här arkitektur det kommer att bli mer stor som Nios II/e.

5. ANVÄNDNING TEST PROGRAMMET I BILAGA A

Se bilaga A.

6. TESTFALLEN ATT SKA ANVÄNDAMED MED DE TVÅ OLIKA PROCESSORERNA(economy och fast)

Testfall 1
Mjukvara (all programvara): Interna FPGA RAM, enable_reduced_device_drivers, enable_small_c_library.

Testfall 2
Mjukvara (all programvara): SRAM minnet/Ingen C-kods optimering

Testfall 3
Mjukvara (all programvara): SRAM minnet/ enable_reduced_device_drivers, enable_small_c_library.

7. TABELL FÖR TESTFALL

Tabell 2: Mätväden från dina testfall

Testfall	Programmets placering	HW Download	
		Economy ingen cashe **HW_CASE_4**	Fast Instruktion 4kb och Data 2kb cashe **HW_CASE_4_F**
1	Interna FPGA RAM/ enable_reduced_device_drivers, enable_small_c_library	offset = 23 time = 158103 tidsbruk = 158080 T = 158103*(1/80) = **1 976 000ns**	offset = 4 time = 7419 tidsbruk = 7415 T = 7415*(1/80) = **92 687,5ns**
2	SRAM minnet/ Ingen C-kods optimering	offset = 53 time = 419241 tidsbruk = 419188 T = 419188*(1/80) = **5 239 850ns**	offset = 4 time = 7509 tidsbruk = 7505 T = 7505*(1/80) = **93 812,5ns**
3	SRAM minnet/ enable_reduced_device_drivers, enable_small_c_library	offset = 53 time = 419408 tidsbruk = 419355 T = 419355*(1/80) = **5 241 937,5ns**	offset = 4 time = 7417 tidsbruk = 7413 T = 7413*(1/80) = **92 662,5ns**

Absolut tid = Antal clock cykler * period tid (system klockan kopplad till timer är 80 MHz).

8. KORFATTAD ANALYS AV SIFFRORNA

Om vi jämföra alla siffrorna som visa tabell 2 vi kan se att fallet HW_CASE_4 utföra alltid programmet mer långsam eftersom den här arkitektur läsa och skriver direkt från huvud minne eftersom har inte cache minne. På HW_CASE_4_F är olika eftersom den här arkitektur är utforma för att använda data cache och instruktioner. HW_CASE_4_F är utforma för att ha en hög prestanda med en olik arkitektur design av HW_CASE_4.

Tabell 3: Analys av siffrorna

Testfall	**Programmets placering**	**HW_CASE_4**	**HW_CASE_4_F**
1	**Interna FPGA RAM**/ enable_reduced_device_drivers, enable_small_c_library	**1 976 000ns:** Minskar koden och data footprint eftersom några kontroller det kompilerar sig med reducerad funktionaliteten. Den har behövt 158080 klokcykel för att utföra hela instruktionerna.	**92 687,5ns:** Minskar koden och data footprint eftersom några kontroller det kompilerar sig med reducerad funktionaliteten. Den har behövt 7415 klokcykel för att utföra alla instruktionerna.
2	**SRAM minnet**/ Ingen C-kods optimering	**5 239 850ns:** Minskar inte koden och data footprint. Den har behövt 419188 klokcykel för att utföra alla instruktionerna.	**93 812,5ns:** Minskar inte koden och data footprint. Den har behövt 7505 klokcykel för att utföra alla instruktionerna.
3	**SRAM minnet/** enable_reduced_device_drivers, enable_small_c_library	**5 241 937,5ns:** Minskar koden och data footprint eftersom några kontroller det kompilerar sig med reducerad funktionaliteten. Den har behövt 419355 klokcykel för att utföra alla instruktionerna.	**92 662,5ns:** Minskar koden och data footprint eftersom några kontroller det kompilerar sig med reducerad funktionaliteten. Den har behövt 7413 klokcykel för att utföra alla instruktionerna.

9. BILAGA

Bilaga A Testprogram

```
#include <stdio.h>
#include <altera_avalon_timer_regs.h>

int main(){
int offset, I;
int value_d,value_c,value_b,value_a,tids_bruk,time;
int sum, results_a, results_b, results_c, results_d;

while(1){

TIMER_RESET;
TIMER_START;
offset=TIMER_READ;

TIMER_RESET;
TIMER_START;

for(I=0;I<100;++I)
{
            value_a=I;
            value_b= I+5;
            value_c=7;
            value_d=7;
            results_a= value_a*145+value_c*346;
            results_b = value_c*value_a*120;
            results_c = value_d+value_b;
            results_d = value_b*value_a+value_c;
            sum = sum + value_b;
}

time=TIMER_READ;
tids_bruk=time-offset;
printf("offset = %d\ntime = %d\ntidsbruk = %d\n" ,offset,time,tids_bruk);
}
return 0;
}
```